Text Sentiment Classification Using Deep Learning (RNN/LSTM)

Assignment 3

Course: Deep Learning Architectures and Techniques

Name: Saksham Semwal
Roll No: 2501940031
Faculty: Dr. Shahid Ahmad Wani

Department of Computer Science and Engineering

School of Engineering and Technology

K.R Mangalam University, Gurugram- 122001, India

1. Objective

The objective of this assignment is to implement deep learning models for sentiment classification using sequential text data. The models used in this assignment include Recurrent Neural Network (RNN), Long Short-Term Memory (LSTM), and Gated Recurrent Unit (GRU). These models help in understanding text sequences and predicting sentiment as positive or negative.

The assignment also focuses on comparing the performance of these models and analyzing how LSTM and GRU improve performance over traditional RNN models.

2. Dataset Description

For this assignment, the IMDB Movie Review dataset is used. This dataset contains movie reviews labeled as positive or negative sentiments.

Dataset Features:

- 50,000 movie reviews
- Binary sentiment classification (Positive/Negative)
- Balanced dataset
- Preprocessed dataset available in TensorFlow

Dataset Split:

- Training Data: 25,000 samples
- Testing Data: 25,000 samples

The dataset is widely used for sentiment classification tasks and helps in evaluating deep learning models for natural language processing problems.

3. Data Preprocessing

Before feeding the data into deep learning models, several preprocessing steps were performed.

Text Cleaning

The following cleaning steps were applied:

- Lowercasing text
- Removing punctuation
- Removing stopwords

- Removing special characters

Tokenization

Tokenization is the process of converting words into numerical values. Each word is assigned a unique integer index.

Example:

"I love this movie"

Converted to:

[45, 23, 87, 102]

Padding

Since each review has different lengths, padding is applied to make sequences equal length.

Maximum Sequence Length = 200

Padding ensures all sequences have the same size, making it easier to train models.

4. Model Architecture

4.1 RNN Model

The RNN model consists of the following layers:

- Embedding Layer
- Simple RNN Layer
- Dense Output Layer

Embedding Layer converts words into dense vectors.

Simple RNN processes sequential text data.

Dense Layer performs classification.

RNN Model Structure:

Embedding Layer (128 units)
Simple RNN (64 units)
Dense Layer (Sigmoid activation)

4.2 LSTM Model

LSTM is an improved version of RNN. It helps capture long-term dependencies.

LSTM uses:

- Forget Gate
- Input Gate
- Output Gate

LSTM Model Structure:

Embedding Layer (128 units)
LSTM Layer (64 units)
Dense Layer (Sigmoid activation)

4.3 GRU Model

GRU is similar to LSTM but simpler. It uses fewer parameters.

GRU uses:

- Update Gate
- Reset Gate

GRU Model Structure:

Embedding Layer (128 units)
GRU Layer (64 units)
Dense Layer (Sigmoid activation)

5. Training Configuration

The following parameters were used:

Epochs: 5
Batch Size: 64
Optimizer: Adam
Loss Function: Binary Crossentropy
Validation Split: 20%

These parameters help in training models efficiently.

6. Implementation Screenshots

```
np.random.seed(42)
tf.random.set_seed(42)
```

```
vocab_size = 10000

(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=vocab_size)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz
17464789/17464789 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training samples: 25000
Testing samples: 25000
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecated. Just
  warnings.warn(
Model: "sequential"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
simple_rnn (SimpleRNN)	?	0 (unbuilt)
dense (Dense)	?	0 (unbuilt)

```
Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)
```

```
lstm_model.add(LSTM(64))
lstm_model.add(Dense(1, activation='sigmoid'))

lstm_model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

lstm_model.summary()
```

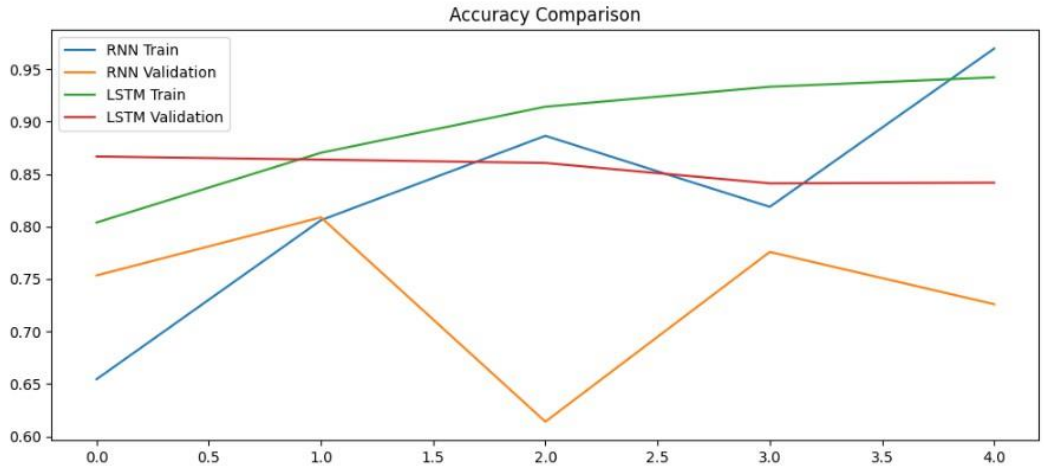
Model: "sequential_2"

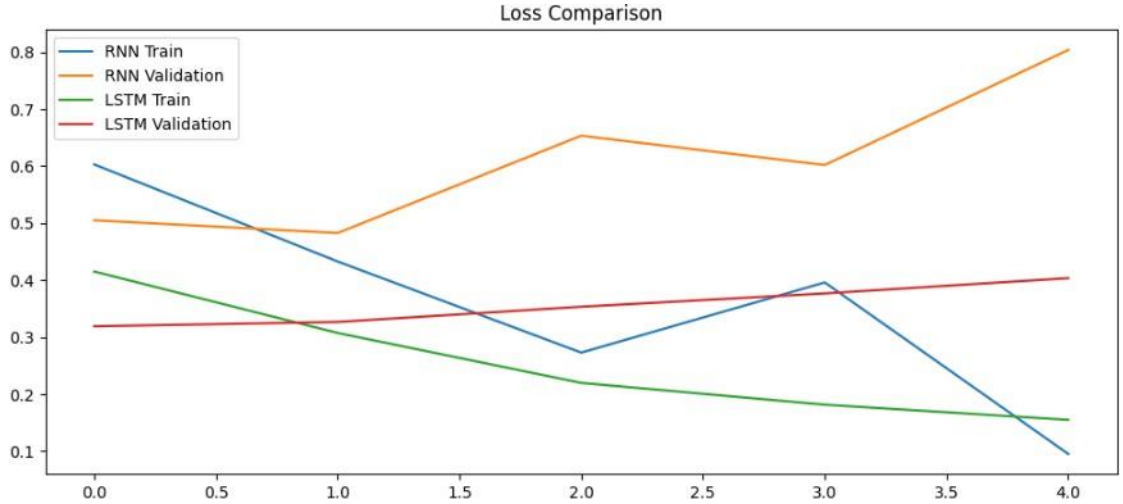
Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	?	0 (unbuilt)
lstm_1 (LSTM)	?	0 (unbuilt)
dense_2 (Dense)	?	0 (unbuilt)

```
Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)
```

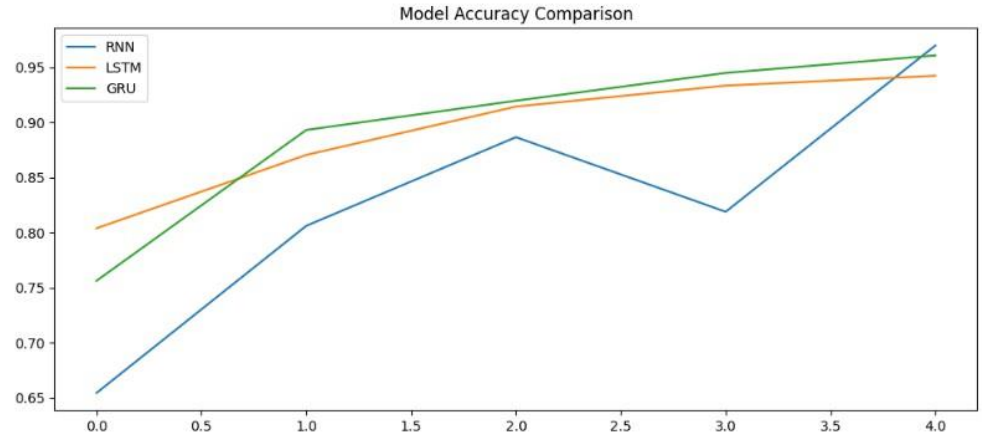
```
print("RNN Report")
print(classification_report(y_test, rnn_pred))

print("LSTM Report")
print(classification_report(y_test, lstm_pred))
```

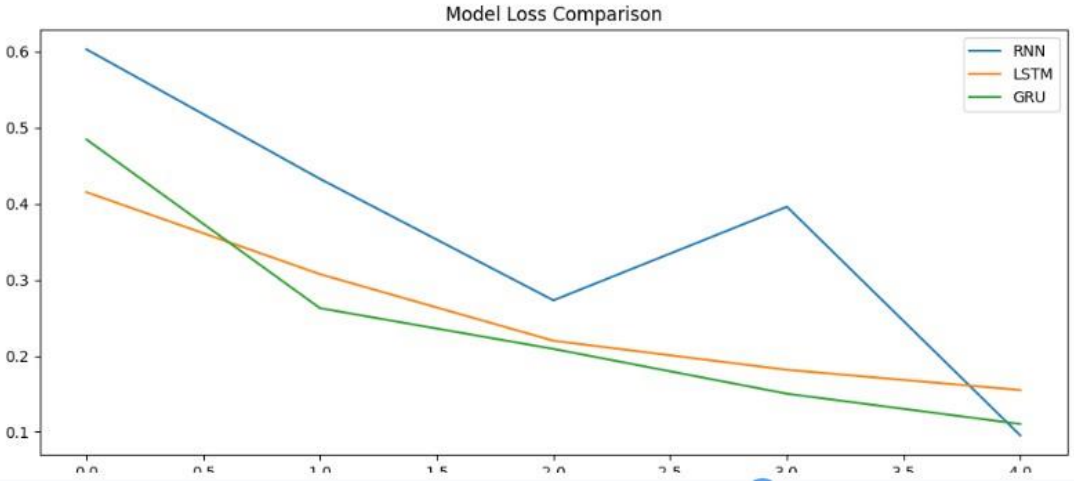
```
RNN Report
              precision    recall  f1-score   support

           0       0.69      0.80      0.74     12500
           1       0.76      0.65      0.70     12500

    accuracy                           0.72     25000
   macro avg       0.73      0.72      0.72     25000
weighted avg       0.73      0.72      0.72     25000

LSTM Report
              precision    recall  f1-score   support

           0       0.86      0.83      0.84     12500
           1       0.83      0.86      0.85     12500

    accuracy                           0.84     25000
   macro avg       0.84      0.84      0.84     25000
weighted avg       0.84      0.84      0.84     25000
```






```
plt.legend()
plt.title("Model Accuracy Comparison")
plt.show()
```




```
plt.figure(figsize=(12,5))
```

```
plt.figure(figsize=(12,5))

plt.plot(history_rnn.history['loss'], label='RNN')
plt.plot(history_lstm.history['loss'], label='LSTM')
plt.plot(history_gru.history['loss'], label='GRU')

plt.legend()
plt.title("Model Loss Comparison")
plt.show()
```